Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τομέας Τεχνολογίας Πληροφορικής και Υπολογιστών
Μάθημα: ″Σχεδιασμός Ενσωματωμένων Συστημάτων″
Ακαδ. Έτος: 2026-2027

1η Εργαστηριακή Άσκηση

Βελτιστοποίηση Αλγορίθμων για Χαμηλή Κατανάλωση Ενέργειας και Υψηλή Απόδοση

Αποστολόπουλος Νικόλαος
(03122614)
Γκριμπογιάννης Χαρίλαος
(03122002)

Περίληψη

Η παρούσα εργασία υλοποιήθηκε στα πλαίσια του μαθήματος ″Σχεδιασμός Ενσωματωμένων Συστημάτων″ και αφορά ...

Περιεχόμενα

1 Βελτιστοποίηση της Εφαρμογής PHODS **3**
1.1 Χαρακτηριστικά του Board 3
1.2 Μέτρηση Χρόνου Εκτέλεσης 4
1.3 Μετασχηματισμοί Κώδικα 4
1.4 Εξερεύνηση Μεγέθους Μπλοκ $B \times B$ 6
1.5 Εξερεύνηση Μεγέθους Μπλοκ $B_x \times B_y$ 8
1.6 Συγκεντρωτικά Αποτελέσματα 11

2 Βελτιστοποίηση της Εφαρμογής tables.c **11**
2.1 Σύγκριση Αρχιτεκτονικών 12
2.2 Loop Unrolling 13
2.3 Μέγεθος L1 Data Cache 13
2.4 Βελτιστοποίηση με Εξαντλητική Αναζήτηση 14
2.5 Βελτιστοποίηση με τη χρήση Γενετικού Αλγορίθμου 16

1 Βελτιστοποίηση της Εφαρμογής PHODS

1.1 Χαρακτηριστικά του Board

Καταγράφουμε τα βασικά χαρακτηριστικά του μικροελεγκτή και του board, συνδυάζοντας το datasheet της οικογένειας Renesas S7G2 και το περιβάλλον e2 studio.

Μνήμες.

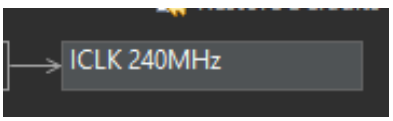
- **RAM (SRAM):** 640 kB συνολικά. Τα πρώτα 32 kB διαθέτουν προστασία Double-bit Error Detection (DED), ενώ η υπόλοιπη περιοχή (608 kB) ελέγχεται με Parity check (Table 1.2 και Table 1.16 του datasheet).
- **Code Flash:** 4 MB, για το part number του board R7FS7G27H3A01CFC (Table 1.15 του datasheet).
- **Data Flash:** 64 kB.

Κρυφές Μνήμες. Το board διαθέτει Flash Cache (FCACHE), η οποία επιταχύνει τις αναγνώσεις από τη μνήμη Flash και καλύπτει την περιοχή διευθύνσεων `0000 0000h–003F FFFFh`. Αποτελείται από τρεις μονάδες:

- **FCACHE1:** εξυπηρετεί τα instruction fetches της CPU. Έχει χωρητικότητα 256 bytes και είναι 8-way set associative, με 2 entries / way και 128 bits / entry.
- **FCACHE2:** εξυπηρετεί τις προσβάσεις της CPU σε δεδομένα (operand access) και τις προσβάσεις από άλλους bus masters, όπως το DMA. Έχει χωρητικότητα μόλις 16 bytes, δηλαδή ένα μόνο entry των 128 bits, και είναι fully associative.
- **FLPF (Prefetch Buffer):** φέρνει εκ των προτέρων τις εντολές που αναμένεται να εκτελεστούν. Έχει χωρητικότητα 32 bytes, δηλαδή 2 entries των 128 bits, και είναι fully associative.

Και στις τρεις μονάδες, ένα cache hit εξυπηρετείται χωρίς κύκλους αναμονής (0 wait states). Σε cache miss, η πρόσβαση στη Flash καθυστερεί τόσους κύκλους αναμονής όσους ορίζει ο Flash Wait Cycle Register.

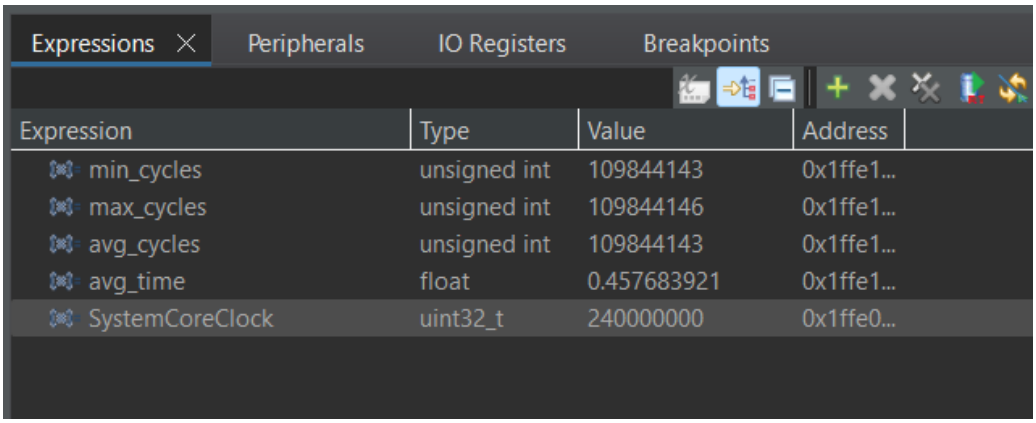
Συχνότητα Ρολογιού. Η συχνότητα του κεντρικού ρολογιού (`SystemCoreClock` / ICLK) είναι 240 MHz. Η τιμή επιβεβαιώθηκε τόσο από τα χαρακτηριστικά μέγιστης λειτουργίας του πυρήνα στο datasheet, όσο και από το e2 studio, όπως φαίνεται στο Σχήμα 1 (καρτέλα Clocks Configuration). Η ίδια τιμή φαίνεται και στο Σχήμα 2, όπου το `SystemCoreClock` δείχνει 240000000.



Σχήμα 1: Συχνότητα ICLK στο Clocks Configuration του e2 studio

1.2 Μέτρηση Χρόνου Εκτέλεσης

Ο χρόνος εκτέλεσης της συνάρτησης `phods_motion_estimation` του αρχικού (μη βελτιστοποιημένου) κώδικα μετρήθηκε διαβάζοντας τον καταχωρητή `DWT->CYCCNT` πριν και μετά την κλήση. Η διαφορά των δύο τιμών δίνει τους κύκλους ρολογιού και διαιρώντας με τη συχνότητα των 240 MHz προκύπτει ο χρόνος. Η διαδικασία επαναλήφθηκε 10 φορές και στο Σχήμα 2 φαίνονται οι τιμές που κατέγραψε ο debugger.



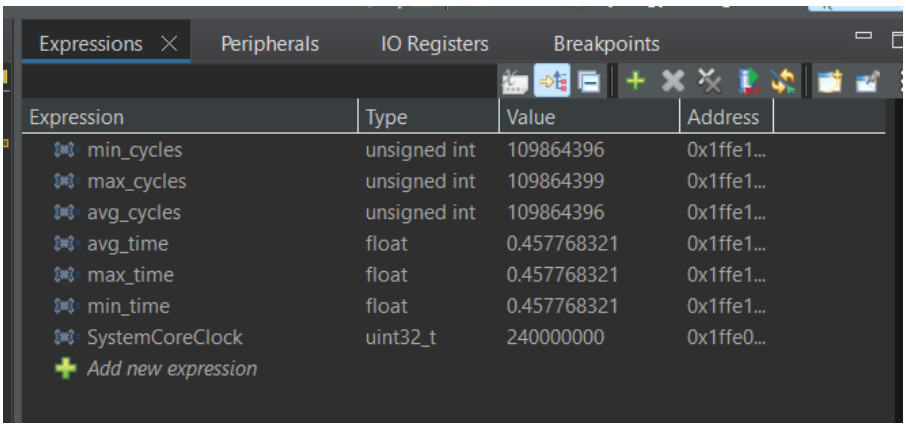
Σχήμα 2: Αποτελέσματα μέτρησης του αρχικού κώδικα (Expressions του e2 studio)

Ο ελάχιστος αριθμός κύκλων είναι 109 844 143 και ο μέγιστος 109 844 146, δηλαδή ο αρχικός κώδικας εκτελείται σε περίπου 0,4577 s. Παρατηρήσαμε ότι μόνο η πρώτη εκτέλεση απαιτεί 109 844 146 κύκλους, ενώ οι υπόλοιπες εννέα απαιτούν 109 844 143. Η ύπαρξη διαφοράς μόνο στην πρώτη εκτέλεση οφείλεται πιθανότατα στην Flash Cache, η οποία είναι ακόμη κενή και προκαλεί επιπλέον cache misses, ενώ από τη δεύτερη εκτέλεση και μετά περιέχει ήδη τα δεδομένα.

1.3 Μετασχηματισμοί Κώδικα

Εφαρμόζουμε διαδοχικά τρεις μετασχηματισμούς στον αρχικό κώδικα και μετράμε κάθε φορά τον χρόνο εκτέλεσης με την ίδια διαδικασία (10 εκτελέσεις). Κάθε μετασχηματισμός εφαρμόζεται πάνω στον κώδικα της προηγούμενης έκδοσης.

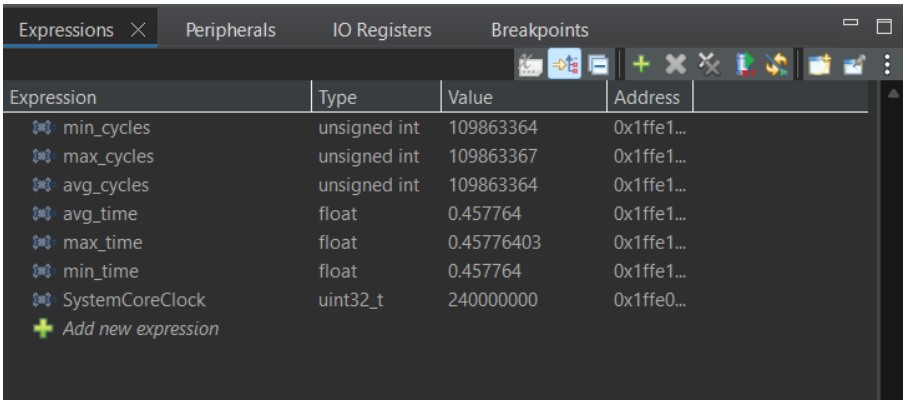
Loop Unrolling. Ο εξωτερικός βρόχος `while(S > 0)` εκτελείται ακριβώς τρεις φορές, με S = 4, 2 και 1. Τον ξετυλίγουμε πλήρως σε τρία διαδοχικά αντίγραφα του σώματός του, με σκοπό την αποφυγή του κόστους ελέγχου και ενημέρωσης του βρόχου. Στο Σχήμα 3 φαίνονται τα αποτελέσματα.



Σχήμα 3: Αποτελέσματα μέτρησης μετά το Loop Unrolling

Ο μετασχηματισμός δεν βελτιώνει την απόδοση, αντίθετα την επιβαρύνει ελάχιστα (+20 253 κύκλοι, περίπου 0,02%). Αυτο συμβαίνει επειδη βρόχος που ξετυλίξαμε εκτελείται μόλις τρεις φορές ανά block, οπότε το κόστος του είναι αμελητέο σε σχέση με το κόστος των εσωτερικών βρόχων. Επιπλέον, ο κώδικας τριπλασιάζεται σε μέγεθος, κάτι που πιθανότατα επιβαρύνει τις προσπελάσεις εντολών από τη Flash.

Loop Reversal. Στους εσωτερικούς βρόχους οι δείκτες k και l μετρούν πλέον αντίστροφα (από B−1 έως 0), ώστε η συνθήκη τερματισμού να γίνεται σύγκριση με το μηδέν, η οποία στον ARM Cortex-M4 είναι φθηνότερη από τη σύγκριση με το B. Δεν εφαρμόσαμε Loop Interchange, καθώς οι πίνακες της C αποθηκεύονται κατά γραμμές και οι υπάρχοντες βρόχοι ήδη τους διατρέχουν γραμμή προς γραμμή, οπότε η εναλλαγή θα κατέστρεφε τη χωρική τοπικότητα. Τα αποτελέσματα φαίνονται στο Σχήμα 4.

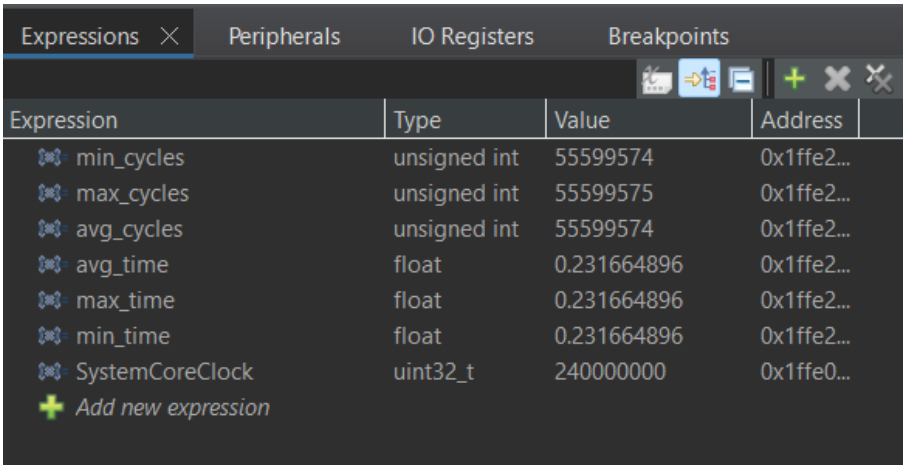


Σχήμα 4: Αποτελέσματα μέτρησης μετά το Loop Unrolling και το Loop Reversal

Ο ελάχιστος αριθμός κύκλων είναι 109 863 364 και ο μέγιστος 109 863 367, δηλαδή 457,764 ms. Σε σχέση με την προηγούμενη έκδοση το κέρδος είναι μόλις 1 032 κύκλοι (περίπου 0,001%), ενώ σε σχέση με τον αρχικό κώδικα ο χρόνος παραμένει λίγο χειρότερος. Η βελτίωση είναι ουσιαστικά αμελητέα, αφού το κόστος εκτέλεσης δεν προέρχεται από τον έλεγχο του δείκτη των βρόχων αλλά από το σώμα τους, δηλαδή από τις προσπελάσεις των πινάκων και τους ελέγχους ορίων.

Data Reuse Transformation (DRT). Οι πίνακες `current` και `previous` είναι `const`, άρα βρίσκονται στη μνήμη Flash, της οποίας οι προσπελάσεις απαιτούν κύκλους αναμονής σε περίπτωση cache miss. Επειδή η Flash Cache του board είναι πολύ μικρή, σχεδόν κάθε

ανάγνωση pixel πληρώνει αυτό το κόστος. Για κάθε block αντιγράφουμε μία φορά σε πίνακες της RAM το block του `current` (B×B) και το παράθυρο αναζήτησης του `previous` ((B+2p)×(B+2p)). Οι υπολογισμοί γίνονται έπειτα αποκλειστικά πάνω στα αντίγραφα της RAM. Κατά την αντιγραφή του παραθύρου τα pixels εκτός εικόνας τίθενται στο 0, οπότε οι έλεγχοι ορίων δεν χρειάζονται πλέον μέσα στον εσωτερικό βρόχο. Στο Σχήμα 5 φαίνονται τα αποτελέσματα.



Σχήμα 5: Αποτελέσματα μέτρησης μετά το Loop Unrolling, το Loop Reversal και το DRT

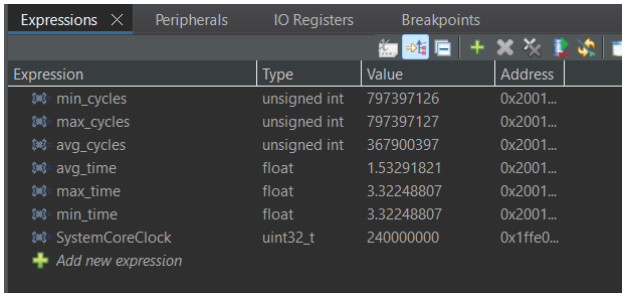
Ο ελάχιστος αριθμός κύκλων είναι 55 599 574 και ο μέγιστος 55 599 575, δηλαδή ο χρόνος εκτέλεσης μειώνεται σε 231,665 ms. Σε σχέση με τον αρχικό κώδικα (457,684 ms) η μείωση είναι περίπου 49%, δηλαδή speedup 1,98×. Η μεγάλη βελτίωση οφείλεται κυρίως στη μεταφορά των δεδομένων από τη Flash στη RAM, καθώς κάθε pixel διαβάζεται πλέον από τη Flash μία φορά ανά block και όχι σε κάθε επανάληψη των υπολογισμών. Αυτό εξηγεί και γιατί οι προηγούμενοι μετασχηματισμοί δεν βελτίωναν την απόδοση: Αυτό εξηγεί και γιατί οι προηγούμενοι μετασχηματισμοί δεν βελτίωναν την απόδοση: αλλάζουν μόνο τη δομή των βρόχων, ενώ το σημείο συμφόρησης ήταν οι προσπελάσεις στη Flash.

1.4 Εξερεύνηση Μεγέθους Μπλοκ $B \times B$

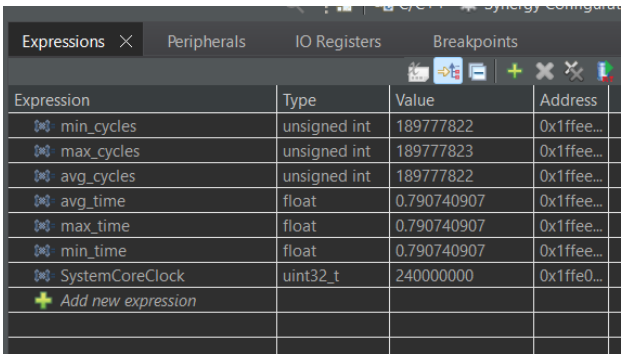
Στον κώδικα του προηγούμενου ερωτήματος (με DRT) εξετάζουμε διαφορετικά μεγέθη τετράγωνου μπλοκ $B \times B$ με εξαντλητική αναζήτηση. Οι εικόνες έχουν διαστάσεις $N = 144$ και $M = 176$, οπότε οι κοινοί διαιρέτες τους, δηλαδή οι τιμές του B που εξετάζουμε, είναι οι 1, 2, 4, 8 και 16.

Σημείωση: Σε αυτό το ερώτημα δηλώσαμε τους πίνακες `motion_vectors_x` και `motion_vectors_y` ως καθολικές (global) μεταβλητές, σε αντίθεση με τα προηγούμενα ερωτήματα, όπου ήταν τοπικές της `hal_entry`. Οι πίνακες αυτοί έχουν ένα στοιχείο για κάθε μπλοκ, οπότε όσο μικραίνει το B μεγαλώνουν. Για $B = 1$ έχουν $144 \times 176 = 25\,344$ στοιχεία ο καθένας (περίπου 99 kB), μέγεθος που δεν χωρά στη στοίβα. Ως global μεταβλητές τοποθετούνται στη RAM εκτός στοίβας. Η ίδια δήλωση χρησιμοποιήθηκε για όλες τις τιμές του B.

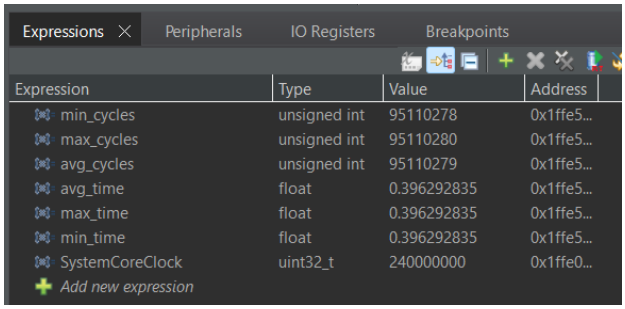
Στο Σχήμα 6 φαίνονται οι τιμές που κατέγραψε ο debugger για κάθε μέγεθος μπλοκ και στον Πίνακα 1 συγκεντρώνονται.



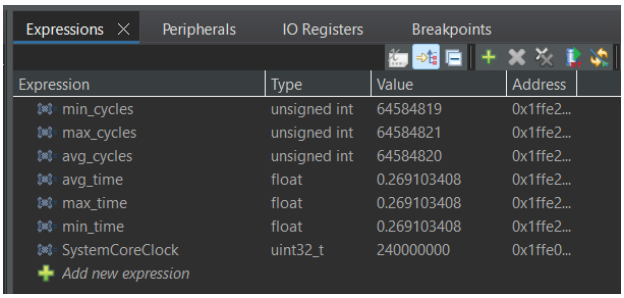
(α') $B = 1$



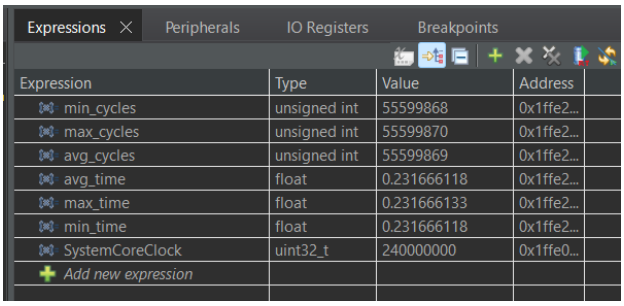
(β') $B = 2$



(γ') $B = 4$



(δ') $B = 8$



(ε') $B = 16$

Σχήμα 6: Αποτελέσματα μέτρησης για διαφορετικά μεγέθη μπλοκ $B \times B$

B	Ελάχιστο (κύκλοι)	Μέγιστο (κύκλοι)	Μέσος όρος (κύκλοι)	Μέσος χρόνος (s)
1	797 397 126	797 397 127	367 900 397*	1,533*
2	189 777 822	189 777 823	189 777 822	0,791
4	95 110 278	95 110 280	95 110 279	0,396
8	64 584 819	64 584 821	64 584 820	0,269
16	55 599 868	55 599 870	55 599 869	0,232

Πίνακας 1: Χρόνος εκτέλεσης για διαφορετικά μεγέθη μπλοκ $B \times B$ (*: λανθασμένη τιμή λόγω υπερχείλισης)

Λανθασμένος μέσος όρος για $B = 1$: Για $B = 1$ ο μέσος όρος που κατέγραψε ο debugger (367 900 397 κύκλοι, δηλαδή 1,533 s) είναι προφανώς λανθασμένος, αφού είναι μικρότερος από τον ελάχιστο αριθμό κύκλων (797 397 126), που ειναι αδύνατο. Η αιτία είναι υπερχείλιση της μεταβλητής `sum_cycles`, η οποία είναι `unsigned int` των 32 bit και μπορεί να αποθηκεύσει τιμές έως $2^{32} - 1 \approx 4{,}29 \cdot 10^9$. Το άθροισμα των 10 εκτελέσεων

είναι περίπου $7{,}97 \cdot 10^9$ κύκλοι, οπότε ξεπερνά το όριο και η μεταβλητή «αναδιπλώνεται», κρατώντας μόνο το $7{,}97 \cdot 10^9 - 2^{32} \approx 3{,}68 \cdot 10^9$. Διαιρώντας με το 10 προκύπτει ακριβώς η λανθασμένη τιμή 367 900 397. Για τις υπόλοιπες τιμές του B το άθροισμα χωρά στα 32 bit και ο μέσος όρος είναι σωστός. Επειδή για $B = 1$ το ελάχιστο και το μέγιστο διαφέρουν μόνο κατά έναν κύκλο, ο πραγματικός μέσος όρος βρίσκεται αναγκαστικά ανάμεσά τους, δηλαδή περίπου 797 397 126 κύκλοι ή 3,322 s, όπως δείχνουν και τα `min_time` και `max_time`. Χρησιμοποιούμε αυτην την τιμη στη σύγκριση.

Ερμηνεία των αποτελεσμάτων. Παρατηρούμε ότι ο χρόνος εκτέλεσης μειώνεται συνεχώς όσο αυξάνεται το B: από 3,322 s για $B = 1$ σε 0,232 s για $B = 16$.

Για να το εξηγήσουμε, χωρίζουμε τη δουλειά του αλγορίθμου σε δύο μέρη:

- **Συγκρίσεις pixels.** Για κάθε μπλοκ ο αλγόριθμος κάνει 3 βήματα αναζήτησης, και σε κάθε βήμα ελέγχει 6 υποψήφιες θέσεις συγκρίνοντας όλα τα B^2 pixels του μπλοκ. Άρα κάθε pixel της εικόνας συγκρίνεται $3 \cdot 6 = 18$ φορές, και συνολικά γίνονται $18 \cdot 144 \cdot 176 = 456\,192$ συγκρίσεις για **οποιοδήποτε** B. Με μικρότερο B έχουμε απλώς περισσότερα και μικρότερα μπλοκ, αλλά η συνολική δουλειά μένει η ίδια.
- **Σταθερό κόστος ανά μπλοκ.** Είναι ό,τι γίνεται μία φορά για κάθε μπλοκ. Το σημαντικότερο είναι η αντιγραφή από τη Flash στη RAM του μπλοκ (B^2 pixels) και του παραθύρου αναζήτησης γύρω του, το οποίο έχει $(B + 14)^2$ pixels.

Αφού οι συγκρίσεις είναι ίδιες για κάθε B, η διαφορά στους χρόνους προέρχεται από το σταθερό κόστος ανά μπλοκ, δηλαδή από την αντιγραφή. Το παράθυρο αναζήτησης έχει πάντα 7 επιπλέον pixels σε κάθε πλευρά, όσο μικρό κι αν είναι το μπλοκ. Για $B = 1$ αντιγράφουμε ένα παράθυρο $15 \times 15 = 225$ pixels για να κάνουμε μόλις 18 συγκρίσεις, ενω για $B = 16$ αντιγράφουμε $30 \times 30 = 900$ pixels, αλλά κάνουμε 4 608 συγκρίσεις. Με μικρά μπλοκ λοιπόν ο επεξεργαστής ξοδεύει τον περισσότερο χρόνο του αντιγράφοντας δεδομένα από τη Flash, ενώ με μεγάλα μπλοκ τον ξοδεύει στις συγκρίσεις.

Το $B = 16$ είναι το ταχύτερο επειδή χωρίζει την εικόνα στα λιγότερα μπλοκ, μόλις $(144/16) \times (176/16) = 9 \times 11 = 99$, και άρα κάνει τις λιγότερες αντιγραφές. Είναι επίσης η μεγαλύτερη τιμή που επιτρέπεται, αφού το 16 είναι ο μέγιστος κοινός διαιρέτης των 144 και 176. Παρατηρούμε ακόμη ότι κάθε διπλασιασμός του B δίνει όλο και μικρότερο κέρδος: ο χρόνος πέφτει 4,20 φορές από $B = 1$ σε 2, αλλά μόλις 1,16 φορές από $B = 8$ σε 16. Αυτό συμβαίνει γιατί για μεγάλα B οι αντιγραφές είναι πια μικρό μέρος του συνολικού χρόνου, και τον περισσότερο χρόνο τον παίρνουν οι συγκρίσεις, που δεν μειώνονται.

Σχέση με τα χαρακτηριστικά του board. Δεν παρατηρούμε σχέση της βέλτιστης τιμής με τις κρυφές μνήμες του board. Οι συγκρίσεις διαβάζουν δεδομένα από τη RAM, την οποία η Flash Cache δεν καλύπτει, ενώ η FCACHE2, που εξυπηρετεί τα δεδομένα της Flash, χωρά μόλις 16 bytes, δηλαδή 4 pixels.

1.5 Εξερεύνηση Μεγέθους Μπλοκ $B_x \times B_y$

Σε αυτό το ερώτημα εξετάζουμε ορθογώνια μπλοκ $B_x \times B_y$, όπου το B_x είναι διαιρέτης του $N = 144$ και το B_y διαιρέτης του $M = 176$. Ο κώδικας είναι ίδιος με του προηγούμενου ερωτήματος, μόνο που το μπλοκ ορίζεται πλέον από δύο σταθερές: το B_x είναι ο αριθμός των **γραμμών** του μπλοκ και το B_y ο αριθμός των **στηλών** του.

Ευριστικό περιορισμού της αναζήτησης. Το 144 έχει 15 διαιρέτες και το 176 έχει 10, άρα υπάρχουν 150 πιθανά ζεύγη. Για να μην τα μετρήσουμε όλα, χρησιμοποιήσαμε τρία κριτήρια:

1. **Όχι πολύ μικρά μπλοκ.** Όπως είδαμε στο ερώτημα 4, για κάθε μπλοκ αντιγράφεται ένα παράθυρο με 7 επιπλέον pixels σε κάθε πλευρά. Σε μικρά μπλοκ το παράθυρο είναι πολύ μεγαλύτερο από το ίδιο το μπλοκ (π.χ. για 4×4 είναι 20 φορές μεγαλύτερο, ενώ για 16×16 μόλις 3,5 φορές). Γι' αυτό απορρίψαμε όλες τις διαστάσεις μικρότερες του 8.

2. **Εμβαδόν κοντά στο** 16×16**.** Αφού το καλύτερο τετράγωνο μπλοκ είχε 256 pixels, εξετάσαμε μπλοκ με εμβαδόν περίπου από 192 έως 384 pixels, αποφεύγοντας πολύ μακρόστενα σχήματα (π.χ. 144×8).

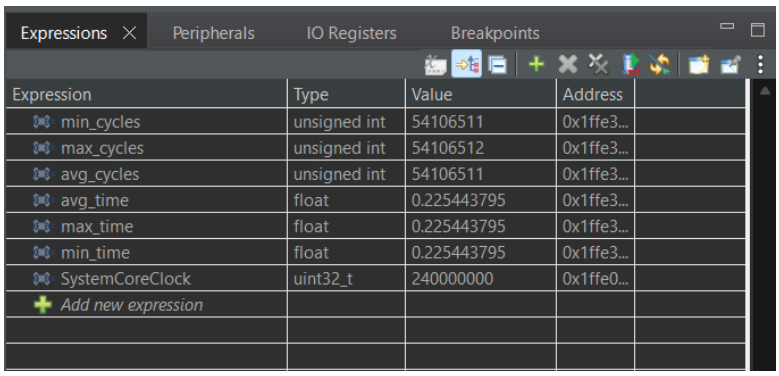
Έτσι μετρήσαμε μόλις 13 από τα 150 ζεύγη. Ανάμεσά τους συμπεριλάβαμε και δύο ζεύγη με ίδιο εμβαδόν αλλά αντίστροφο σχήμα, τα (16, 8) – (8, 16) και (24, 11) – (12, 22), ώστε να δούμε αν παίζει ρόλο ο προσανατολισμός του μπλοκ.

Αποτελέσματα. Στον Πίνακα 2 φαίνονται τα αποτελέσματα, μαζί με τον συνολικό αριθμό στοιχείων που αντιγράφονται από τη Flash στη RAM. Για σύγκριση περιλαμβάνεται και το 16×16 του ερωτήματος 4. Σε όλες τις μετρήσεις το ελάχιστο και το μέγιστο διαφέρουν το πολύ κατά 4 κύκλους, οπότε δίνουμε μόνο τον μέσο όρο.

(B_x, B_y)	Εμβαδόν	Μέσος όρος (κύκλοι)	Μέσος χρόνος (s)	Αντιγραφές από Flash
Σταθερό $B_y = 8$				
(9, 8)	72	61 985 694	0,2583	203 456
(12, 8)	96	59 775 427	0,2491	176 352
(16, 8)	128	59 282 735	0,2470	156 024
(18, 8)	144	57 338 934	0,2389	149 248
(24, 8)	192	56 118 403	0,2338	135 696
(36, 8)	288	54 906 168	0,2288	122 144
Σταθερό $B_y = 16$				
(8, 16)	128	59 235 730	0,2468	156 024
(9, 16)	144	58 113 991	0,2421	146 784
(12, 16)	192	56 007 036	0,2334	128 304
(16, 16)	256	55 599 869	0,2317	114 444
(18, 16)	288	54 944 575	0,2289	109 824
(24, 16)	**384**	**54 106 511**	**0,2254**	**100 584**
Ίδιο εμβαδόν, αντίστροφος προσανατολισμός				
(24, 11)	264	57 229 841	0,2385	116 544
(12, 22)	264	54 332 194	0,2264	115 200

Πίνακας 2: Χρόνος εκτέλεσης για ορθογώνια μπλοκ $B_x \times B_y$

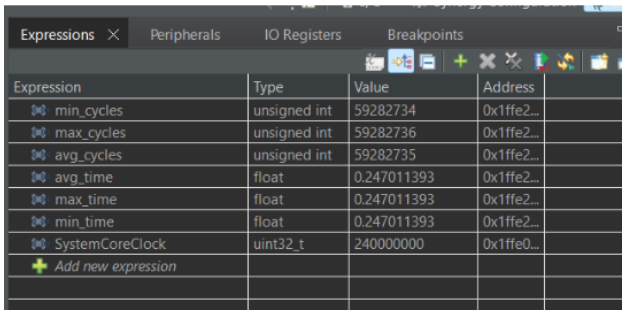
Το καλύτερο ζεύγος είναι το $(B_x, B_y) = (24, 16)$, με 54 106 511 κύκλους (0,2254 s). Οι τιμές του debugger φαίνονται στο Σχήμα 7.



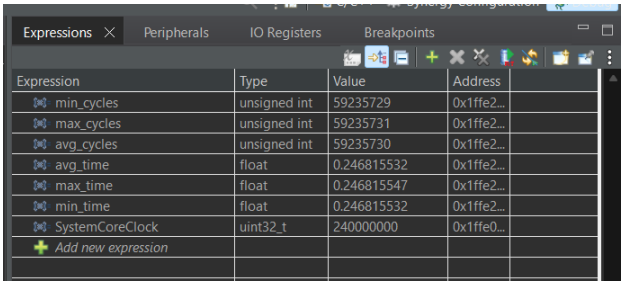
Σχήμα 7: Αποτελέσματα μέτρησης για το βέλτιστο μπλοκ 24×16

Ερμηνεία: μέγεθος του μπλοκ. Ισχύει το ίδιο με το ερώτημα 4: όσο μεγαλύτερο είναι το μπλοκ, τόσο λιγότερα μπλοκ έχει η εικόνα και τόσο λιγότερα στοιχεία αντιγράφονται από τη Flash. Γι' αυτό ο χρόνος μειώνεται καθώς μεγαλώνει το B_x, και για το ίδιο B_x το $B_y = 16$ είναι πάντα ταχύτερο από το $B_y = 8$. Το $(24, 16)$ έχει τις λιγότερες αντιγραφές (100 584) και είναι το ταχύτερο. Είναι όμως μόλις 2,7% ταχύτερο από το 16×16, γιατί σε τόσο μεγάλα μπλοκ οι αντιγραφές είναι ήδη μικρό μέρος του συνολικού χρόνου.

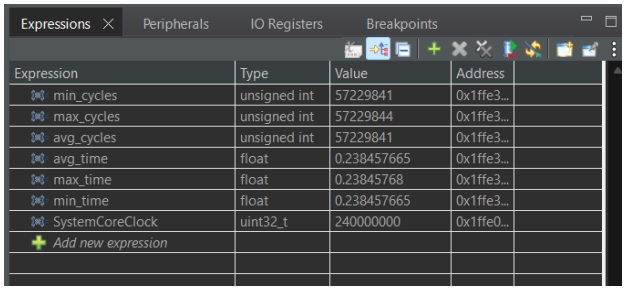
Ερμηνεία: προσανατολισμός του μπλοκ. Οι τιμές του debugger για τα μπλοκ με ίδιο εμβαδόν φαίνονται στο Σχήμα 8.



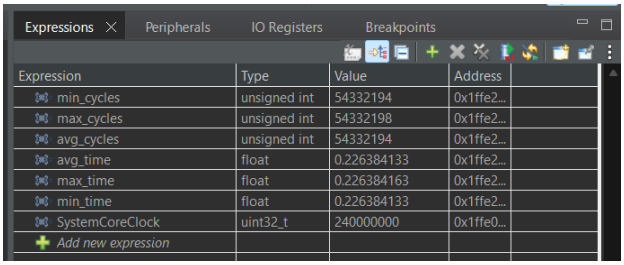
(α') $(B_x, B_y) = (16, 8)$



(β') $(B_x, B_y) = (8, 16)$



(γ') $(B_x, B_y) = (24, 11)$



(δ') $(B_x, B_y) = (12, 22)$

Σχήμα 8: Αποτελέσματα μέτρησης για μπλοκ ίδιου εμβαδού με αντίστροφο προσανατολισμό

Η C αποθηκεύει τους πίνακες κατά γραμμές, οπότε ένα «πλατύ» μπλοκ (μεγάλο B_y) αντιγράφεται από τη Flash σε λίγες μακριές γραμμές, ενώ ένα «ψηλό» (μεγάλο B_x) σε πολλές κοντές. Επειδή η FCACHE2 κρατά μόνο μία εγγραφή, κάθε αλλαγή γραμμής προκαλεί σίγουρα miss, άρα τα πλατιά μπλοκ θα πρέπει να είναι λίγο ταχύτερα.

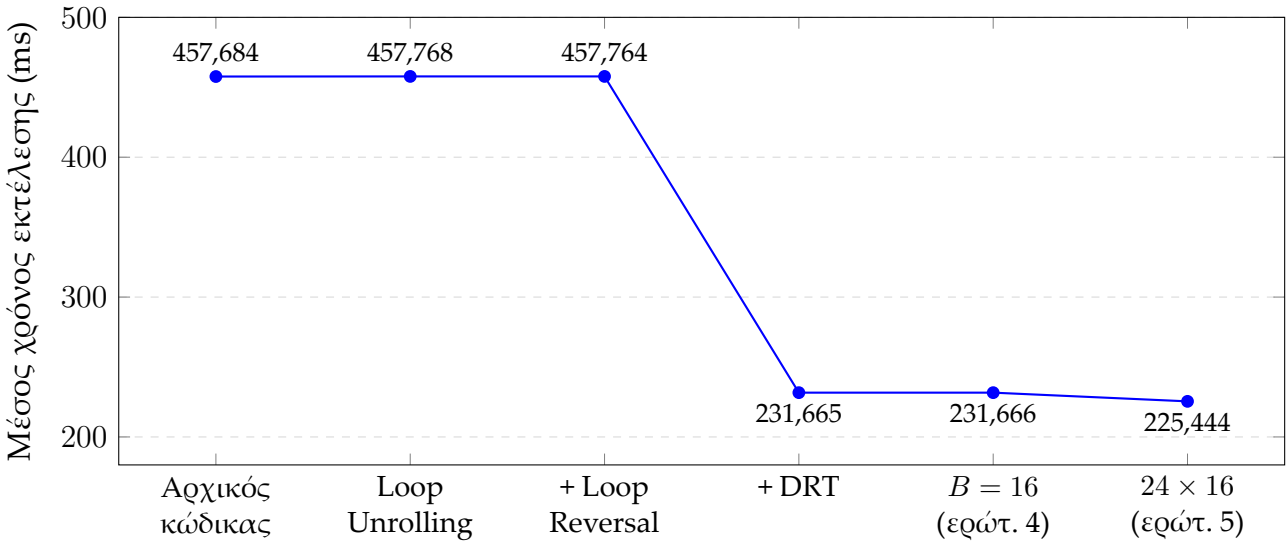
Τα δύο μπλοκ του ζεύγους (16, 8) – (8, 16) αντιγράφουν ακριβώς τα ίδια στοιχεία, όμως το πλατύ (8, 16) αντιγράφεται σε λιγότερες γραμμές και άρα με λιγότερα misses. Πράγματι είναι ταχύτερο, αλλά μόλις κατά 47 005 κύκλους (0,08%). Επομένως ο προσανατολισμός παίζει πολύ μικρότερο ρόλο από το μέγεθος του μπλοκ.

Στο ζεύγος (24, 11) – (12, 22) το πλατύ (12, 22) είναι επίσης ταχύτερο, αλλά κατά 5,3%. Η διαφορά αυτή είναι πολύ μεγαλύτερη από όσο θα περιμέναμε μόνο από τον προσανατολισμό, και δεν μπορούμε να την εξηγήσουμε με βάση τις μετρήσεις μας.

Αξιολόγηση του ευριστικού. Το κριτήριο των μικρών μπλοκ επιβεβαιώνεται, ενώ το κριτήριο του εμβαδού όχι: τα μπλοκ των 288 και 384 pixels ήταν ταχύτερα από το 16×16, το οποίο ήταν το καλύτερο στο ερώτημα 4 μόνο επειδή ήταν το μεγαλύτερο επιτρεπτό τετράγωνο. Επειδή δεν εξετάσαμε μπλοκ μεγαλύτερα από 384 pixels, το $(24, 16)$ είναι το καλύτερο από όσα μετρήσαμε και όχι απαραίτητα το βέλτιστο όλου του χώρου.

1.6 Συγκεντρωτικά Αποτελέσματα

Στο Σχήμα 9 φαίνεται ο μέσος χρόνος εκτέλεσης για όλες τις εκδόσεις του κώδικα, με τη σειρά που τις υλοποιήσαμε. Για τα ερωτήματα 4 και 5 δίνεται η καλύτερη διάσταση μπλοκ που βρήκαμε.



Σχήμα 9: Μέσος χρόνος εκτέλεσης σε όλες τις εκδόσεις του κώδικα (ερωτήματα 2–5)

Το διάγραμμα δείχνει ότι σχεδόν όλη η βελτίωση οφείλεται στο DRT. Το Loop Unrolling και το Loop Reversal αφήνουν τον χρόνο πρακτικά αμετάβλητο, ενώ το DRT τον μειώνει στο μισό. Η εξερεύνηση του μεγέθους μπλοκ δεν βελτίωσε περαιτέρω τον χρόνο στο ερώτημα 4, αφού το $B = 16$ ήταν ήδη η τιμή του αρχικού κώδικα. Στο ερώτημα 5 το ορθογώνιο μπλοκ 24×16 έδωσε μια μικρή επιπλέον μείωση 2,7%. Συνολικά, ο χρόνος εκτέλεσης μειώθηκε από 457,684 ms σε 225,444 ms, δηλαδή speedup περίπου 2,03× σε σχέση με τον αρχικό κώδικα.

2 Βελτιστοποίηση της Εφαρμογής tables.c

Στην παρούσα παράγραφο βελτιστοποιούμε την εφαρμογή tables.c (βλ. Listing 1) εκτελούμενη σε δύο διαφορετικές αρχιτεκτονικές x86 με χρήση του προσομοιωτή Gem5.

Η πρώτη αρχιτεκτονική που μελετάμε αφορά μια αρχιτεκτονική επεξεργαστή και κύριας μνήμης, ενώ η δεύτερη είναι επαυξημένη με L1 Instruction Cache, L1 Data Cache και L2 Cache. Συγκεκριμένα, μελετάμε την επίδραση της τεχνικής του loop unrolling, καθώς και την επίδραση της ύπαρξης κρυφών μνημών διαφορετικών μεγεθών στον χρόνο εκτέλεσης της εφαρμογής μετρημένο σε κύκλους ρολογιού.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;
    int N = 100000;
    double *x1, *x2, *x3, *y;
    double a1 = 0.5;
    double a2 = 1;
    double a3 = 1.5;

    x1 = (double*) malloc(N * sizeof(double));
    x2 = (double*) malloc (N * sizeof(double));
    x3 = (double*) malloc (N * sizeof(double));
    y = (double*) malloc (N * sizeof(double));

    //Do not modify this loop
    for (i=0; i<=N-1; i++)
    {
        x1[i] = (double) i * 0.5;
        x2[i] = (double) i * 0.8;
        x3[i] = (double) i * 0.2;
        y[i] = 0;
    }

    for (i=0; i<=N-1; i++)
    {
        y[i] = y[i] + a1*x1[i] + a2*x2[i] + a3*x3[i];
    }

    return 0;

}
```

Listing 1: Source Code (tables.c)

2.1 Σύγκριση Αρχιτεκτονικών

Αρχικά προσομοιώνουμε την εκτέλεση της εφαρμογής tables.exe σε αρχιτεκτονική που περιλαμβάνει τον επεξεργαστή και την κύρια μνήμη. Η εκτέλεση γίνεται σε 858,490,254 κύκλους. Ύστερα προσομοιώνουμε την εκτέλεση της ίδιας εφαρμογής σε σύστημα που διαθέτει επιπλέον L1 Instruction και L1 Data Caches μεγέθους 8kB και L2 Cache μεγέθους 128kB. Η εφαρμογεί εκτελείται σε 59,829,295 κύκλους. Παρατηρούμε ότι η ενσωμάτωση κρυφών μνημών στο σύστημα μείωσε τον χρόνο εκτέλεσης κατά 798,660,959 κύκλους δίνοντας ένα speedup περίπου 14.35x. Η δραματική αυτή αύξηση της ταχύτητας εκτέλεσης αποδεικνύει έμπρακτα την ανάγκη ύπαρξης των κρυφών μνημών, καθώς χωρίς αυτές κάθε ανάγνωση/εγγραφή που εκτελεί το πρόγραμμα απαιτεί πολύ αργές προσπελάσεις στην κύρια μνήμη.

2.2 Loop Unrolling

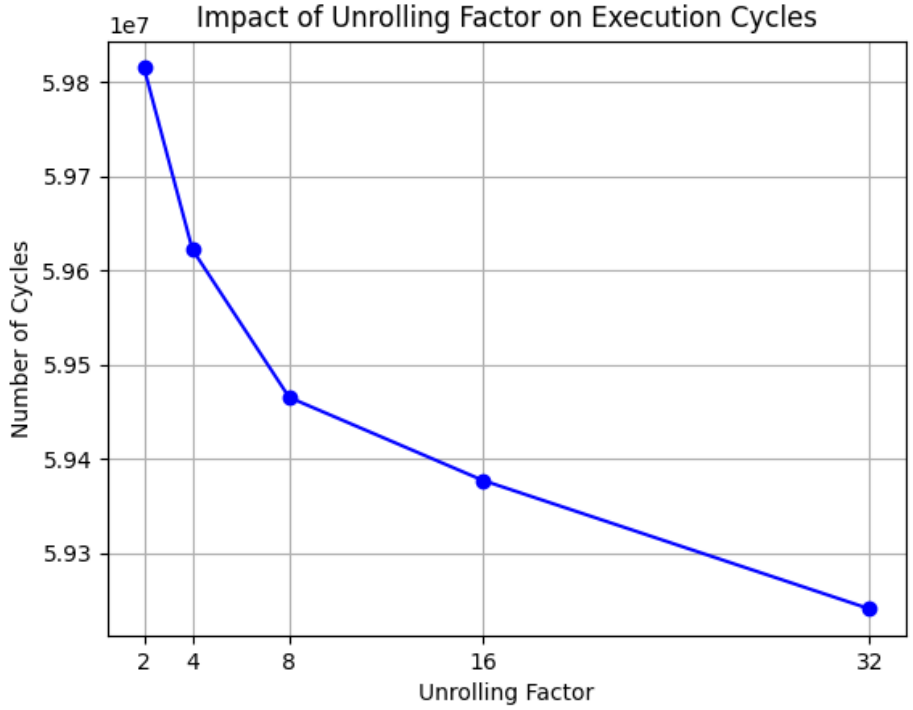
Αφού αποδείξαμε την ανωτερότητα της αρχιτεκτονικής με κρυφές μνήμες, θα μελετήσουμε πλέον μόνο αυτή την αρχιτεκτονική. Για να ελαττώσουμε περαιτέρω τον χρόνο εκτέλεσης της εφαρμογής, εισάγουμε τη μέθοδο loop unrolling και εξετάζουμε την επίδραση της εφαρμογής της.

Η μέθοδος loop unrolling είναι ένας μετασχηματισμός που συνίσταται στο "ξετύλιγμα" του βρόχου. Στο παρόν την εφαρμόζουμε στον βρόχο υπολογισμού του πίνακα y όπως φαίνεται στο Listing 2.

```
for (i = 0; i <= N - k; i = i + k) {
    y[i] = y[i] + a1 * x1[i] + a2 * x2[i] + a3 * x3[i];
    ...
    y[(i + k)] = y[(i + k)] + a1 * x1[(i + k)] + a2 * x2[(i + k)] + a3 * x3[(i + k)];
}
for (int i = N - ((N - (0)) % k); i <= N - 1; i = i + 1)
    y[i] = y[i] + a1 * x1[i] + a2 * x2[i] + a3 * x3[i];
```

Listing 2: Loop Unrolling with Unrolling Factor k

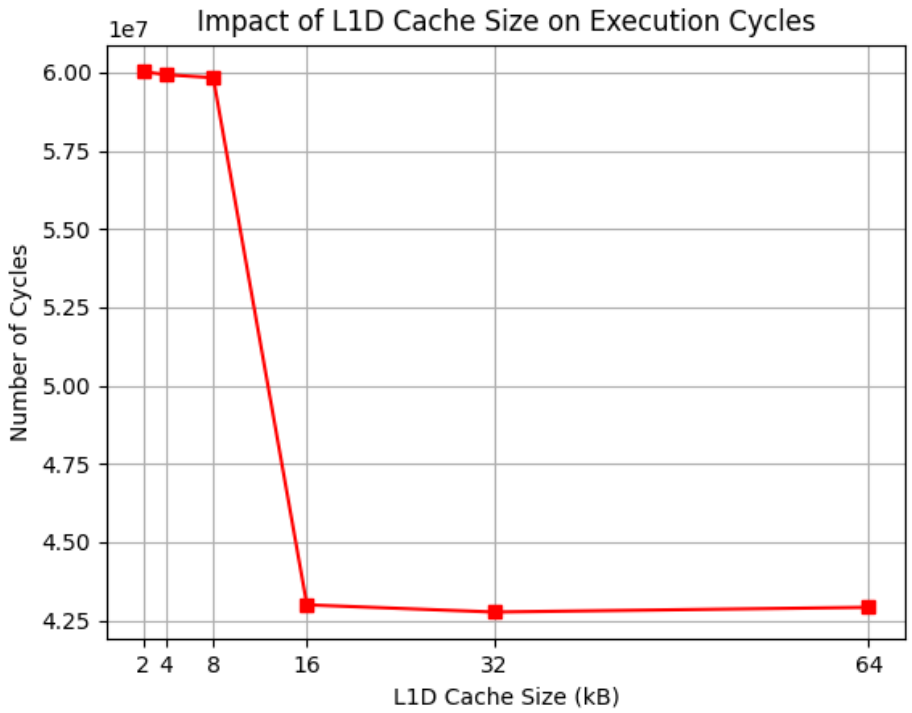
Στο Σχήμα 10 φαίνεται η επίδραση του μετασχηματισμού στον χρόνο εκτέλεσης σε πλήθος κύκλων ρολογιού για διαφορετικά unrolling factors (2, 4, 8, 16 και 32). Παρατηρούμε ότι ... Αυτό οφείλεται...



Σχήμα 10: Επίδραση του Unrolling Factor στον Χρόνο Εκτέλεσης

2.3 Μέγεθος L1 Data Cache

Συνεχίζοντας τη μελέτη της δεύτερης αρχιτεκτονικής, προχωράμε σε μια εξερεύνηση της επίδρασης διαφορετικών μεγεθών L1 Data Cache (2kB, 4 kB, 8kB, 16kB, 32kB και 64kB).



Σχήμα 11: Επίδραση του Μεγέθους της L1 Data Cache στον Χρόνο Εκτέλεσης

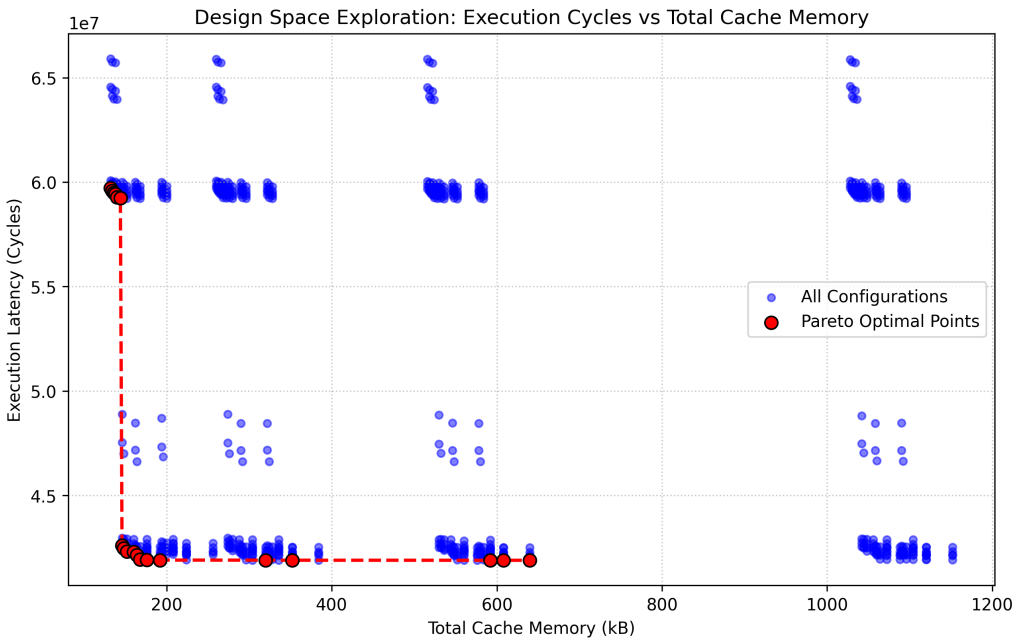
Εδώ παρατηρούμε κάτι πολύ ενδιαφέρον...

2.4 Βελτιστοποίηση με Εξαντλητική Αναζήτηση

Έχοντας μελετήσει την επίδραση συγκεκριμένων παραμέτρων της εφαρμογής (Unrolling Factor) αλλά και της αρχιτεκτονικής (Μέγεθος L1 Data Cache), έχουμε σχηματίσει μια εικόνα για τη βελτιστοποίηση. Αυτό που μας ενδιαφέρει είναι να ελαχιστοποιήσουμε το πλήθος των κύκλων που απαιτούνται για την εκτέλεση της εφαρμογής αλλά και τη συνολική μνήμη που θα χρησιμοποιήσουμε (το άθροισμα των L1I, L1D και L2 σε kB). Έτσι, καταγράφουμε έναν χώρο προς εξερεύνηση, ο οποίος ορίζεται από τους συνδυασμούς των παρακάτω

L1D Cache Size $\in$ [2kB, 4kB, 8kB, 16kB, 32kB, 64kB]
L1I Cache Size $\in$ [2kB, 4kB, 8kB, 16kB, 32kB, 64kB]
L2 Cache Size $\in$ [128kB, 256kB, 512kB, 1024kB]
Unrolling Factor $\in$ [2, 4, 8, 16, 32]

Προχωράμε σε εξαντλητική αναζήτηση του χώρου εξερεύνησης και για κάθε ένα από τα 720 σημεία του χώρου καταγράφουμε το πλήθος των κύκλων που απαιτούνται για την εκτέλεση της εφαρμογής. Στο Σχήμα 12 φαίνονται οι θέσεις όλων των σημείων και το Pareto Frontier, ενώ στον Πίνακα 3 παραθέτουμε το Pareto Frontier μαζί με τα στοιχεία της αρχιτεκτονικής που οδήγησαν στο εκάστοτε Pareto Optimal σημείο.



Σχήμα 12: Exhaustive Search Pareto Frontier

L1I (kB)	L1D (kB)	L2 (kB)	UF	Total Memory (kB)	Latency (CC)
2	2	128	8	132	59724828
2	16	128	8	146	42591707
4	2	128	16	134	59579732
4	4	128	16	136	59484646
4	16	128	16	148	42472280
4	32	128	16	164	42146735
8	2	128	32	138	59427801
8	4	128	32	140	59281539
8	8	128	32	144	59241019
8	16	128	32	152	42315832
8	32	128	32	168	41928215
16	16	128	32	160	42300132
16	32	128	32	176	41912388
16	64	512	32	592	41889209
32	32	128	32	192	41898540
32	32	256	32	320	41896021
32	64	256	32	352	41892206
32	64	512	32	608	41889106
64	64	512	32	640	41888238

Πίνακας 3: Exhaustive Search Pareto Optimal Points

Παρατηρούμε ότι... Τέλος, σημειώνουμε ότι λόγω του μεγάλου πλήθους προσομοιώσεων (720 σημεία), για την εξαντλητική αναζήτηση απαιτήθηκαν 2 ώρες και 46 λεπτά.

2.5 Βελτιστοποίηση με τη χρήση Γενετικού Αλγορίθμου

Αντί για τη χρονοβόρα εξαντλητική αναζήτηση του χώρου εξερεύνησης, μπορούμε να χρησιμοποιήσουμε έναν γενετικό αλγόριθμο για να λάβουμε το Pareto Frontier. Ο αλγόριθμος αυτός μπλα μπλα μπλα...

Εκτελώντας τον αλγόριθμο παρατηρούμε ότι κάνει μόνο 30 evaluations και τερματίζει σε 7 λεπτά και 30 δευτερόλεπτα, επιστρέφοντας το Pareto Frontier του Πίνακα 4. Παρατηρούμε αρχικά ότι ο αλγόριθμος βρήκε 3 Pareto Optimal σημεία, σε αντίθεση με τα 19 της εξαντλητικής αναζήτησης. Τέλος, παρατηρούμε ότι μόνο ένα εκ των τριών σημείων (32kB, 32kB, 256kB, 32) είναι στην πραγματικότητα Pareto Optimal και βρίσκεται στα αποτελέσματα της προηγούμενης υποπαραγράφου. Αυτό συμβαίνει καθώς ο αλγόριθμος βασίζεται σε έναν ευρετικό (heuristic) μηχανισμό...

L1I (kB)	L1D (kB)	L2 (kB)	UF	Total Memory (kB)	Latency (CC)
32kB	32kB	512kB	32	576	41895278
16kB	32kB	128kB	16	176	42130415
32kB	32kB	256kB	32	320	41896021

Πίνακας 4: Genetic Algorithm Pareto Optimal Points